AI ACADEMY · LEVEL 5 · GRADE 10 · WEEK 32 · STUDENT LESSON

Documenting a network experiment

Learn to understand, design, build, test and explain AI systems responsibly.

Project: Image-pattern Network Investigation

THE CONNECTED WEEK

Record the inputs. Retain the history. Bound the conclusion.

Replay a tiny neural experiment and trace every reported metric to its data role and saved state.

Keep one habit: Run question → data manifest → complete checkpoint ledger → bounded result → separate picture → artifact checklist → executable replay → independent audit.

MAIN LESSON · SECTION 1

Build a record another person can replay

> A score becomes checkable when its inputs, parameters and calculation are preserved.

Learning goals

I can record a network experiment with enough detail to **replay** its calculations.

I can trace reported metrics to data roles and saved parameters.

I can write a bounded conclusion and identify missing evaluation **evidence**.

Retrieve measurement and evidence

Close your notes first. A rough first answer helps us choose the right starting point; it is not a score for your ability.

Without opening the old lesson, explain one key idea from Efficiency and resource measurement. Give an example and a mistake that the idea helps you avoid.

Without opening the old lesson, explain one key idea from Uncertainty and failure analysis. Give an example and a mistake that the idea helps you avoid.

Without opening the old lesson, explain one key idea from Embeddings and neighbourhoods. Give an example and a mistake that the idea helps you avoid.

After trying, check the relevant lesson or ask your teacher. Change the part of your explanation that the **evidence** shows was wrong.

Prerequisite and starting response

- Why is “the model was fast and accurate” insufficient as an experiment record?

> **My starting explanation**
>
> _____
>
> _____
>
> _____
>
> _____

End goal

By the end, I can explain documenting a network experiment using a traced example and a stated limitation.

▼ **Earlier wording, alternatives and picture transcripts**

Your learning target

Remember before reading

This week you will practise:

Record a network experiment with enough detail to **replay** its calculations

Trace reported metrics to data roles and saved parameters

Write a bounded conclusion and identify missing evaluation **evidence**

Retrieve what you already know

This week’s end goal

MAIN LESSON · SECTION 2

Define run NN32-A before computing its result

> Record the question, data roles, starting weight and selection rule.

Question and constructed-data scope

A useful experiment record explains what was asked, what was done, what happened and what remains unknown.

Use run NN32-A: does **fitting** one positive ReLU weight reduce validation mean squared error compared with its initial value?

This tiny constructed regression task teaches record keeping; it cannot establish performance on real-world data.

Complete training and validation manifest

Data role and ID	Input x	Target y
Training T1	1	2
Training T2	2	4
Validation V1	3	6

Architecture, update settings and environment

The network predicts max(0, a*x), with one weight a and no bias. Start at a = 1, use learning rate 0.1, two full-batch **training** updates and mean squared error, the mean of squared **prediction** errors.

There is no normalisation, random initialisation or shuffling. Use Python float arithmetic and record the actual Python version when **replay**ing.

Baseline, selection rule and unmeasured final evaluation

Before running, set the selection rule: choose the **checkpoint** with the smaller validation MSE, breaking a tie toward the earlier update. The initial model is the baseline.

Validation V1 chooses a **checkpoint** but never contributes to the **training** gradient. Final evaluation has not been run; its result must be recorded as not measured.

Name supplied and missing facts

> **List the given facts and one fact that is not supplied.**

MAIN LESSON · SECTION 3

Connect each update to its prediction rows

> Use training rows for gradients and validation only for checkpoint selection.

Derive both restricted gradients and updates

For these two **training** cases, MSE = 2.5*(a - 2) squared and its gradient is 5*(a - 2). At a = 1 the gradient is -5, so the first update is 1 - 0.1*(-5) = 1.5. The next gradient is -2.5, giving a = 1.75.

All preactivations in this run stay positive, so ReLU's derivative is one; this is not a general backpropagation implementation.

Complete baseline and checkpoint ledger

State · Gradient before update · Weight · Train predictions T1/T2 · Training MSE · V1 prediction · Validation MSE

> **State:** Initial baseline
> **Gradient before update:** Not yet updated
> **Weight:** 1
> **Train predictions T1/T2:** 1 / 2
> **Training MSE:** 2.5
> **V1 prediction:** 3
> **Validation MSE:** 9

> **State:** Update 1
> **Gradient before update:** -5
> **Weight:** 1.5
> **Train predictions T1/T2:** 1.5 / 3
> **Training MSE:** 0.625
> **V1 prediction:** 4.5
> **Validation MSE:** 2.25

> **State:** Update 2
> **Gradient before update:** -2.5
> **Weight:** 1.75
> **Train predictions T1/T2:** 1.75 / 3.5
> **Training MSE:** 0.15625
> **V1 prediction:** 5.25

> **Validation MSE:** 0.5625

Original runnable two-update calculation

```
train = [(1.0, 2.0), (2.0, 4.0)]
validation = [(3.0, 6.0)]

def predict(a, x):
    return max(0.0, a*x)

def mse(a, rows):
    return sum((predict(a, x)-y)**2 for x, y in rows) / len(rows)

a, rate = 1.0, 0.1
baseline = mse(a, validation)
records = []
for update in [1, 2]:
    gradient = sum(2*(a*x-y)*x for x, y in train) / len(train)
    a -= rate * gradient
    records.append((update, a, mse(a, train), mse(a, validation)))

chosen = min(records, key=lambda r: (r[3], r[0]))
print(baseline, records, chosen[0])
```

Positive-preactivation assumptions and retained history

The code assumes the supplied nonempty numeric datasets and positive preactivations. Each record stores update number, weight, **training** MSE and validation MSE after the update.

Keep both records, the initial weight and the rule; saving only the best score erases how the choice was made.

Explain the visual

Point to the **input**, the deciding step and the result in this week's visual. Explain the same step in ordinary words. A picture helps when you can say what each part represents.

▼ Earlier wording, alternatives and picture transcripts

Connect the picture and the explanation

MAIN LESSON · SECTION 4

Select a saved checkpoint without renaming its evidence

> The validation case is part of development because it chooses the checkpoint.

Current task and response

- Calculate both weight updates and give each **training** and validation MSE. Name the selected **checkpoint** and baseline validation MSE. How many independent validation examples support the comparison?

> **Keep the procedure fixed. Explain what changed in the inputs and result.**

Predict the changed picture claim

- Change: A teammate calls V4 a final test after using it to select a=1.25.

▼ Check after predicting

Prediction: Correct its role to validation; it was used for selection.

Reason: Changing the label on a dataset does not make its **evidence** untouched.

▼ Earlier wording, alternatives and picture transcripts

Compare a familiar case with a changed case

Change one thing in the pictured case

MAIN LESSON · SECTION 6

Write a conclusion that names the run and data role

> A lower validation MSE on one constructed case is a bounded result.

Main checkpoint comparison and defensible conclusion

Read each checkpoint against the same validation case

CHECKPOINT WEIGHT	TRAINING MSE	VALIDATION MSE
Update 1: a = 1.5	0.625	2.25
Update 2: a = 1.75	0.15625	0.5625

The baseline validation MSE is 9. The predeclared rule selects update 2 with validation MSE 0.5625 on one validation case. This is not a final-test score.

A defensible conclusion is: in run NN32-A, update 2 reduced MSE on the single constructed validation case from 9 to 0.5625 compared with the initial model.

This **checkpoint** was selected using that same validation case; generalisation is untested. No timing, peak-memory or energy measurements are provided.

Try the selected prediction and role explanation

> **Using the selected weight, calculate the prediction for V1 and its squared error. Explain why the result must be labelled validation rather than independent final evaluation.**

Separate pictured run and full givens

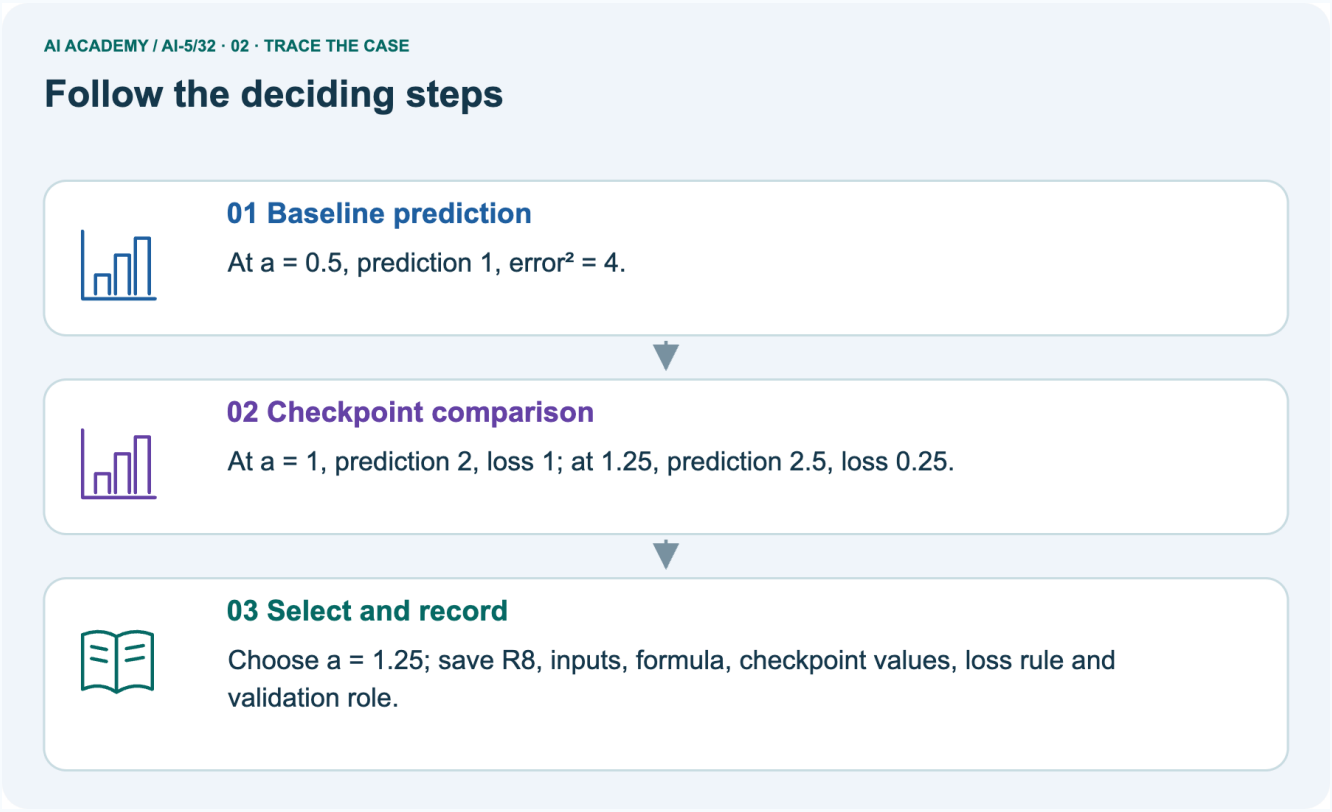
Document **checkpoint** selection so another team can reconstruct every reported number.

Run R8 predicts ReLU(a×x). Validation V4 has x = 2, target 3. Baseline a = 0.5; **checkpoint**s a = 1 and 1.25; choose minimum validation squared error.

All **input**s and outcomes in this worked example are invented classroom data; no real model run is claimed.

Main NN32-A uses V1 x = 3, target 6: baseline validation MSE 9, then 2.25 and .5625. The picture is a separate run R8 using V4 x = 2, target 3: supplied weights .5, 1, 1.25 give predictions 1, 2, 2.5 and squared errors 4, 1, .25. It selects weight 1.25, but no training procedure for those pictured checkpoints is supplied. Both examples use validation to choose; neither provides independent final evaluation.

Trace the pictured checkpoint choice



Follow the labelled picture with the matching explanation. Open the vector diagram

Read the picture one step at a time

STEP	WHAT HAPPENS AND WHY
1. Baseline prediction	At a = 0.5, prediction 1, error² = 4.
2. Checkpoint comparison	At a = 1, prediction 2, loss 1; at 1.25, prediction 2.5, loss 0.25.
3. Select and record	Choose a = 1.25; save R8, inputs, formula, checkpoint values, loss rule and validation role.

Match each pictured step to the supplied facts. This is a worked teaching case, not an independent assessment.

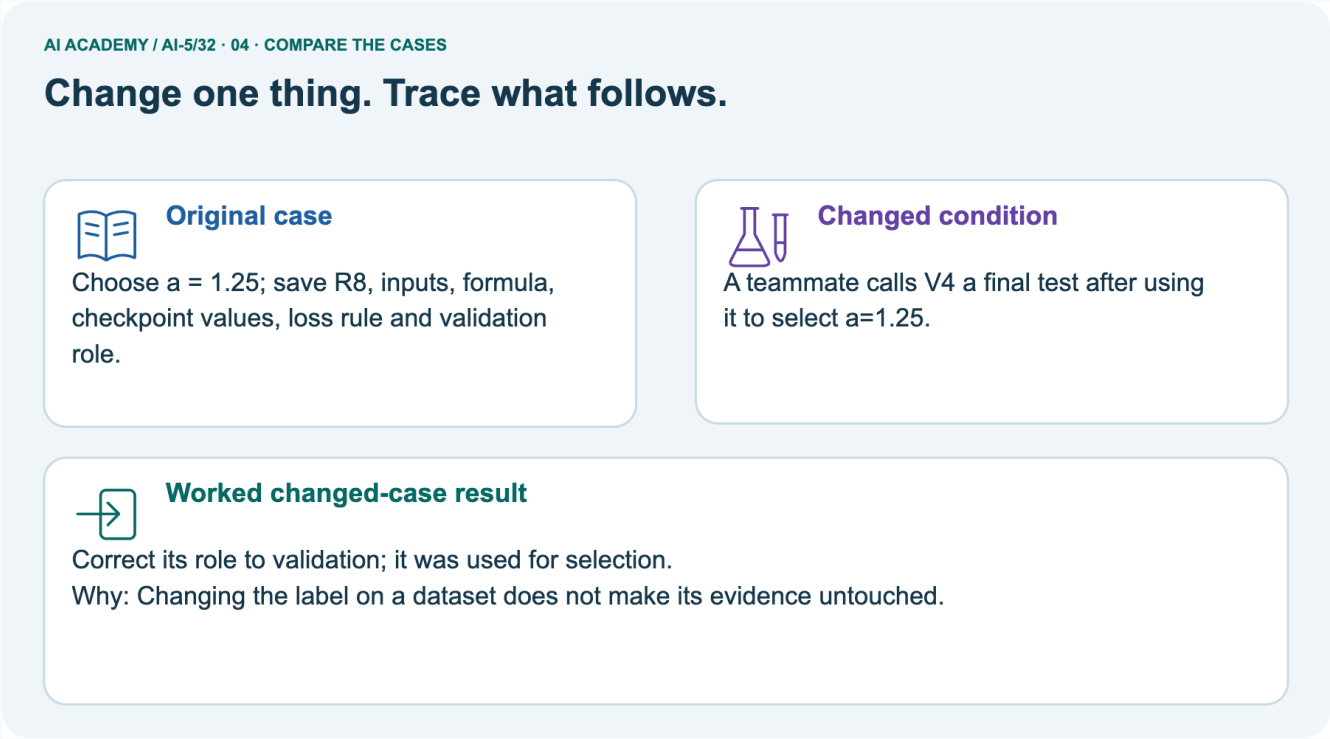
A reproducible record includes the actual **input**, formula, parameters and scoring rule, together with each dataset's role. The validation case selects the best supplied **checkpoint**, so it is part of development.

Save that selection history and freeze the **checkpoint** before evaluating fresh final cases. Renaming the same case a test does not change how it was used.

Predict the effect of renaming validation

- Change one thing: A teammate calls V4 a final test after using it to select a=1.25. Predict the result before checking the explanation. What remains the same?

▼ Check after predicting



Follow the labelled picture with the matching explanation. Open the vector diagram

Explain additional evidence and remaining limits

- Reconstructing one run verifies arithmetic and provenance, not population performance or educational learning impact.

- Read the labelled picture: What additional **evidence** is needed for a final performance claim? Point to the relevant step in the picture, then write the changed result and the rule or **evidence** that supports it.

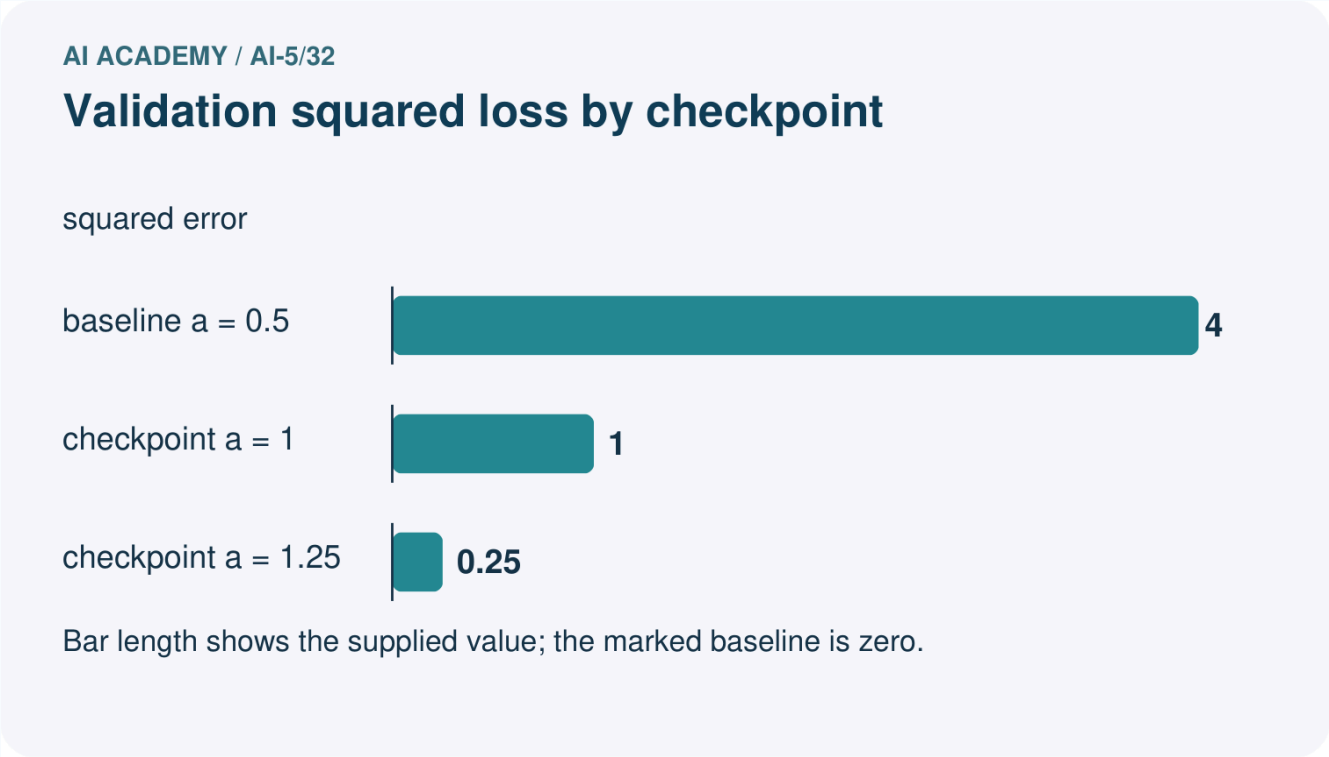
▼ Earlier wording, alternatives and picture transcripts

Learn from the worked case

Cover the result before checking it. Say what is given, choose the procedure, and follow one step at a time. Then uncover the result and explain your first mismatch. Ask why a step is allowed, not only what number it produces.

A pictorial worked case: Documenting a network experiment

The facts we will use



Follow the labelled picture with the matching explanation. Open the vector diagram

AI ACADEMY / AI-5/32

Documenting a network experiment

Start with the supplied facts

Follow each arrow. Ask which fact allows the next step.

1. Baseline prediction

At a = 0.5, prediction 1, error2 = 4.

2. Checkpoint comparison

At a = 1, prediction 2, loss 1; at 1.25, prediction 2.5, loss 0.25.

3. Select and record

Choose a = 1.25; save R8, inputs, formula, checkpoint values, loss rule and validation role.

CHANGE ONE THING

A teammate calls V4 a final test after using it to select a=1.25.

Explain the changed result using the matching lesson.

Follow the labelled picture with the matching explanation. Open the vector diagram

Why this result follows

What this example does not establish

Keep the evidence chain intact

> Hashes identify content versions; they do not certify a good experiment.

Manifest, exact versions and discrepancy handling

Keep a **manifest** for the data with IDs and split membership, the exact code, settings, every **checkpoint**, raw **prediction** rows and metric calculations.

Add run ID, author/date, environment versions, known data provenance and permitted use, failed attempts, decisions and limitations.

A descriptive filename helps navigation, but does not prove that file contents stayed unchanged.

When saving artifacts, record their content hashes and keep the exact versions used. Hashes help detect content changes; they do not prove label correctness or experiment quality.

Retain the **input** order, any **training**-fitted pre**process**ing, model architecture and numeric precision.

Randomised experiments must also record seeds and the relevant random operations; a seed alone does not guarantee identical results across environments.

Ask a partner to **replay** from the record rather than from memory. If results differ, check **input**s, split membership, pre**process**ing, configuration, code, parameters and environment.

Document the discrepancy and the repair instead of overwriting the old record. A rerun on the same data checks the procedure; it does not add new independent evaluation cases.

Artifact-to-purpose checklist

Artifact or record	Why another person needs it
Data manifest: T1/T2 training, V1 validation	Recover exact inputs, targets, IDs, roles and ordering
Exact code and configuration	Recover model formula, loss, rate, initial state, update count and assumptions
Baseline and every checkpoint	Reconstruct the comparison and selection history

Artifact or record	Why another person needs it
Raw predictions and metric calculations	Trace each reported squared error to a case and weight
Python/environment versions and actual run status	Separate a real replay from a paper trace and record possible differences
Content hashes for exact saved versions	Detect changed content; do not prove reference correctness or study quality
Final evaluation, time, peak memory, energy	Record not measured when absent; do not replace with zero
Changed data or preprocessing	Preserve original artifacts and record a revised run rather than overwrite history

Diagnose the incomplete folder

- A folder contains only best-model.json and a screenshot of 0.5625. Identify four missing artifacts or records and explain what a content hash can and cannot verify.

State one repair and the checks you would rerun.

▼ **Earlier wording, alternatives and picture transcripts**

Locate the earliest unsupported step

Practise tracing a result back to its source

> Write the update, dataset role and denominator beside each metric.

Current task and guided calculation

- For NN32-A, record the **prediction** rule, initial weight, bias policy, learning rate, update count, loss, **training**/validation IDs, data order or shuffling policy, and **checkpoint** selection rule.

  Name one environment field still to record when you run it.

> **Using the selected weight, calculate the prediction for V1 and its squared error. Explain why the result must be labelled validation rather than independent final evaluation.**

> **Explain your prediction to a partner before comparing results.**

Supported W2 record

Use workbook W2 for the connected example “Documenting a network experiment”. First fill the **input** and rule boxes. Try the middle steps before asking for a hint. After a hint, try the next step yourself.

Plan	Do	Check
What is given? Which rule or procedure applies?	Record each intermediate step.	Does the result follow? What remains unknown?

Explain your trace to a partner. Your partner points to one step to check. Swap roles on the next case. Each person writes their own explanation.

Individual record-quality check

- Use the worked case for Documenting a network experiment. Proposed explanation: “A smaller MSE is an accuracy percentage”. Decide whether this explanation is supported.

  Point to the step or supplied fact that decides; explain your repair if needed.

- Answer on your own before discussion. If you are unsure, say which step you need help with.

Pictured evidence question

- What additional **evidence** is needed for a final performance claim? Point to the relevant step in the picture, then write the changed result and the rule or **evidence** that supports it.

My explanation using a pictured step and the given facts

Reserve independent R17

This is supported practice. Keep the separately named independent case for an attempt before feedback.

▼ **Earlier wording, alternatives and picture transcripts**

Practise with less help: Trace the selected result

Check your reasoning

Explain the picture

MAIN LESSON · SECTION 9

Replay and inspect a complete record

> The generated record distinguishes actual environment information from supplied classroom data.

Current update and metric investigation

Calculate both weight updates and give each **training** and validation MSE. Name the selected **checkpoint** and baseline validation MSE. How many independent validation examples support the comparison?

Complete executable replay and in-memory record

```
"""Replay the supplied toy run; emit an in-memory record, not a final test."""
import hashlib
import json
import platform

def content_hash(value):
    payload=json.dumps(value,sort_keys=True,separators=(',',':'),allow_nan=False)
    return hashlib.sha256(payload.encode('utf-8')).hexdigest()

manifest=[{'id':'T1','role':'training','x':1.0,'target':2.0},
          {'id':'T2','role':'training','x':2.0,'target':4.0},
          {'id':'V1','role':'validation','x':3.0,'target':6.0}]
config={'run_id':'NN32-A','architecture':'ReLU(a*x); no bias',
        'initial_weight':1.0,'learning_rate':.1,'full_batch_updates':2,
        'loss':'mean squared error','preprocessing':'none',
        'random_operations':'none',
        'selection':'minimum validation MSE, earlier update wins ties'}

def predict(a,x):
    return max(0.0,a*x)

def mse(a,rows):
    if not rows:
        raise ValueError('MSE needs at least one row')
    return sum((predict(a,r['x'])-r['target'])**2 for r in rows)/len(rows)

def replay(data,config):
```

```
    training=[r for r in data if r['role']=='training']
    validation=[r for r in data if r['role']=='validation']
    if not training or not validation:
        raise ValueError('Need separate training and validation rows')
    if len({r['id'] for r in data})!=len(data):
        raise ValueError('Duplicate data ID')
    a=config['initial_weight']
    baseline={'weight':a,'training_mse':mse(a,training),
              'validation_mse':mse(a,validation)}
    checkpoints=[]
    for update in range(1,config['full_batch_updates']+1):
        # This deliberately restricted gradient requires all training a*x > 0.
        if any(a*r['x']<=0 for r in training):
            raise ValueError('Outside the supplied positive-preactivation gradient')
        gradient=sum(2*(a*r['x']-r['target'])*r['x'] for r in training)/len(training)
        a-=config['learning_rate']*gradient
        predictions=[{'id':r['id'],'role':r['role'],
                      'prediction':predict(a,r['x']),
                      'squared_error':(predict(a,r['x'])-r['target'])**2}
                     for r in data]
        checkpoints.append({'update':update,'gradient_before':gradient,'weight':a,
                            'training_mse':mse(a,training),
                            'validation_mse':mse(a,validation),
                            'prediction_rows':predictions})
    selected=min(checkpoints,key=lambda r:(r['validation_mse'],r['update']))
    return {'baseline':baseline,'checkpoints':checkpoints,
            'selected_update':selected['update'],
            'selected_weight':selected['weight']}

result=replay(manifest,config)
assert result['baseline']['validation_mse']==9
assert [(r['weight'],r['training_mse'],r['validation_mse'])
        for r in result['checkpoints']]==[(1.5,.625,2.25),(1.75,.15625,.5625)]
assert result==replay(manifest,config)  # replay, not new independent evidence
record={'config':config,'data_manifest':manifest,'data_sha256':content_hash(manifest),
        'config_sha256':content_hash(config),'python_version':platform.python_version(),
        'result':result,'final_evaluation':'not measured','runtime':'not measured',
        'peak_memory':'not measured','energy':'not measured',
        'provenance':'constructed classroom regression rows'}
print(json.dumps(record,indent=2))
# A hash detects changed serialized content; it does not establish label quality.
changed=[dict(r) for r in manifest]
changed[-1]['target']=7.0
assert content_hash(changed)!=content_hash(manifest)
assert content_hash(manifest)==record['data_sha256']
print('Changed validation target detected; original manifest retained.')
# Save the exact code separately with its own hash when creating an actual run folder.
```

Predictions, scope, content checks and unmeasured fields

1. Predict both updates and metric rows before running or tracing the complete replay. It emits a JSON record in memory with the actual Python version when run, complete data and settings, raw checkpoint predictions, content hashes and not-measured fields. It does not save files or claim a human replay occurred.

   The restricted training gradient rejects nonpositive training preactivations; it is not a general ReLU trainer. The script demonstrates that changing the validation target changes its content hash while preserving the original manifest. Save the exact script separately with its own hash when making an actual experiment folder. Replaying the same rows checks arithmetic, not independent final performance.

Evidence ledger and reflection

Record	Your evidence
Given input and fixed rule	
Prediction before checking	
Actual result, or “not run”	
First mismatch and one explanation	
Boundary or missing input and expected response	

On paper, trace the supplied procedure. If code is provided, predict before running it in a local practice file. Never write an expected answer as though it were an observed run.

Before trying: what do I expect, and why? While trying: which step could first go wrong? Afterwards: what did I actually observe, and what should change? Keep “not run” if you only traced the procedure.

▼ Reference: earlier two-update snippet

```
train = [(1.0, 2.0), (2.0, 4.0)]
validation = [(3.0, 6.0)]
```

```
def predict(a, x):
    return max(0.0, a*x)

def mse(a, rows):
    return sum((predict(a, x)-y)**2 for x, y in rows) / len(rows)

a, rate = 1.0, 0.1
baseline = mse(a, validation)
records = []
for update in [1, 2]:
    gradient = sum(2*(a*x-y)*x for x, y in train) / len(train)
    a -= rate * gradient
    records.append((update, a, mse(a, train), mse(a, validation)))

chosen = min(records, key=lambda r: (r[3], r[0]))
print(baseline, records, chosen[0])
```

▼ Earlier wording, alternatives and picture transcripts

Predict, investigate and preserve the evidence

Plan → try → check

MAIN LESSON · SECTION 14

Try alone: audit the fresh R17 checkpoints

> Compute the new predictions and errors before writing a bounded conclusion.

Complete independent run and data

A separate run R17 records baseline a = 1 and **checkpoint**s a = 1.5 and a = 1.75. It uses **prediction** max(0, a*x) and one validation case V9 with x = 4, y = 8.

Compute baseline and **checkpoint prediction**s and squared errors, choose by lowest validation MSE, and write a bounded conclusion. Final evaluation and runtime have not been measured.

First attempt and support record

> **My independent explanation, trace and limits**
>
> | | |
> |---|---|
> | | |
> | | |
> | | |
> | | |

Attempt this case before feedback. Record any help. Earlier examples below are further practice; a case already practised with feedback cannot establish fresh transfer.

Feedback after trying

Attempt the independent case before hints. Record whether you worked alone, used a hint or followed a model. After feedback, fix the first unsupported step and explain why it changed.

A later check uses a changed case, not a copied answer.

▼ Earlier wording, alternatives and picture transcripts

Independent case: Audit a new record independently

Attempt this without hints

A separate run R17 records baseline a = 1 and **checkpoint**s a = 1.5 and a = 1.75. It uses **prediction** max(0, a*x) and one validation case V9 with x = 4, y = 8.

Compute baseline and **checkpoint prediction**s and squared errors, choose by lowest validation MSE, and write a bounded conclusion. Final evaluation and runtime have not been measured.

My independent procedure, result and limitation

Record support used: none, hint or teacher explanation. After feedback, a different case is needed to demonstrate independence.

Use feedback to improve

MAIN LESSON · SECTION 11

Save a replayable project record

> Link the question to the exact data, model states, predictions and conclusion.

Project task and artifact

Add a labelled documenting a network experiment **evidence** sheet to Image-pattern Network Investigation. Show how you can record a network experiment with enough detail to **replay** its calculations.

Use the supplied fictional case and keep its **input**s, intermediate steps, calculation or **prediction**, and limitation. Connect this record to last week's record; a matching answer without a trace is insufficient.

Project evidence	Record
Component and version	
Inputs permitted at prediction time	
Trace and comparison with earlier work	
Failure, limit and human review	

Actual run or trace and limitations

For your Image-pattern Network Investigation, save the **input**, procedure, observed result and limitation of this check.

Label this worked example as practice; use a different, previously unreviewed case when checking independent understanding.

▼ **Earlier wording, alternatives and picture transcripts**

Evidence for your project

MAIN LESSON · SECTION 16

Finish with a checkable result and an honest next step

> Preserve missing evidence as missing and keep the original run intact.

Retrieve and explain

- Close your notes. Explain this goal in your own words: Record a network experiment with enough detail to **replay** its calculations

My deciding step, evidence and one remaining question

Plan a changed-case return

If you are unsure, return to the worked picture and explain one arrow. If you are ready, change an **input** or assumption and justify your **prediction**.

Save your workbook **evidence**. At the next review, try a different case before opening your old answer.

Reference bank and extra practice

Choose the reference that helps your next step.

▼ **Reference · Use the mathematical words precisely**

REFERENCE · SECTION 5

Reference · Use the mathematical words precisely

> Connect each term to a value or step in the current example.

Vocabulary and your explanation

Word	Meaning
Run identifier	A unique label linking one experiment’s settings, outputs and records
Checkpoint	Saved model parameters at a specified stage of training
Manifest	An inventory connecting named artifacts with versions, roles and integrity information
Replay	Repeating a recorded procedure to check whether its outputs agree

Use two terms to explain a step in this week’s example.

▼ Extra challenge · Explain a stronger case

REFERENCE · SECTION 10

Extra challenge · Explain a stronger case

> Choose the next challenge after the main route.

Explain,repair and extend

- For NN32-A, record the **prediction** rule, initial weight, bias policy, learning rate, update count, loss, **training**/validation IDs, data order or shuffling policy, and **checkpoint** selection rule.

  Name one environment field still to record when you run it.

- A folder contains only best-model.json and a screenshot of 0.5625. Identify four missing artifacts or records and explain what a content hash can and cannot verify.

- A partner gets different results after changing pre**process**ing and replacing the validation file. They keep the old run ID and overwrite the old score.

  Explain what should be preserved and how the revised work should be recorded.

Explain → apply → challenge

> **Explain which evidence supports your conclusion and which claim remains unproved.**

▼ Reference · Check the representation and its limits

REFERENCE · SECTION 12

Reference · Check the representation and its limits

> Use after practice;reserve the separate independent run R17 for the first individual attempt.

Current explanation and response

- Report missing measurements as not measured, not as zero. If a final set has influenced a change, document that use and reserve new untouched evaluation before making an independent performance claim.

  A polished report cannot fix leakage or missing **evidence**. For larger experiments, preserve denominators, subgroup results and resource conditions from Weeks 29 to 31.

- Finish with a next action that follows from the limitations, such as planning a broader reserved evaluation without claiming it has already succeeded.

  Next week begins the capstone by defining a hypothesis and baseline before controlled experiments.

- A partner gets different results after changing pre**process**ing and replacing the validation file. They keep the old run ID and overwrite the old score.

  Explain what should be preserved and how the revised work should be recorded.

A report says “final accuracy 0.5625, low energy” but includes only the selected weight and no data or code. Repair its claim and name the minimum evidence needed to check this run.

My limited claim and a useful next check

▼ **Extra practice · Revisit the complete trace**

REFERENCE · SECTION 13

Extra practice · Revisit the complete trace

> Predict before reading the supplied steps.

Complete trace and evidence

Complete evidence record

Read each checkpoint against the same validation case

CHECKPOINT WEIGHT	TRAINING MSE	VALIDATION MSE
Update 1: a = 1.5	0.625	2.25
Update 2: a = 1.75	0.15625	0.5625

The baseline validation MSE is 9. The predeclared rule selects update 2 with validation MSE 0.5625 on one validation case. This is not a final-test score.

Follow the **input** through the stated procedure to its result. Cover the result first, predict it, then explain every intermediate step. The text/table accompanies the figure so the example can be read without colour.

> **My complete trace of the worked case**

▼ **Reference · The earlier route checklist**

REFERENCE · SECTION 15

Reference · The earlier route checklist

> Use one main route rather than repeating the lesson.

Targets,route and first prediction

Start with this question: Document **checkpoint** selection so another team can reconstruct every reported number.

By the end, I can	How I will show it
Record a network experiment with enough detail to replay its calculations	Explain the deciding step, show a trace or test, and state one limit.
Trace reported metrics to data roles and saved parameters	Explain the deciding step, show a trace or test, and state one limit.
Write a bounded conclusion and identify missing evaluation evidence	Explain the deciding step, show a trace or test, and state one limit.

Remember → watch and explain → try with help → investigate → try independently → keep project **evidence**. Your teacher may spread this lesson across several classes.

Keep your first **prediction**. Compare it with what the **evidence** shows and explain any change.

▼ **Extra practice · Predict,check and repair**

REFERENCE · SECTION 17

Extra practice · Predict,check and repair

> Keep supported feedback separate from the independent case.

First supported question and optional hint

- Document **checkpoint** selection so another team can reconstruct every reported number.
- Which labelled step matches this detail? Choose a = 1.25; save R8, **input**s, formula, **checkpoint** values, loss rule and validation role.
- 1. **Checkpoint** comparison
- 2. Select and record
- 3. Baseline **prediction**
- Hint if needed: Read the steps in the pictorial case. Match the supplied detail to the stage that performs or records it.

Practice 1: explain the deciding evidence

> **My selected conclusion, deciding fact and explanation**

▼ **Check after the first attempt**

After your attempt, compare with the supported explanation: Select and record Choose a = 1.25; save R8, **input**s, formula, **checkpoint** values, loss rule and validation role.

A reproducible record includes the actual **input**, formula, parameters and scoring rule, together with each dataset's role. The validation case selects the best supplied **checkpoint**, so it is part of development.

Save that selection history and freeze the **checkpoint** before evaluating fresh final cases. Renaming the same case a test does not change how it was used.

Changed-case question and optional hint

- Changed-case practice: A teammate calls V4 a final test after using it to select a=1.25. Which conclusion is supported by the supplied facts?
- 1. We can decide the result without examining the changed **input** or the procedure.
- 2. This one example guarantees correct results for every future **input**.
- 3. Correct its role to validation; it was used for selection.
- Hint if needed: Use the changed condition and the deciding step in the pictorial trace. The answer must say what follows from those facts.

Practice 2: explain the deciding evidence

My selected conclusion, deciding fact and explanation

▼ **Check after the changed attempt**

After your attempt, compare with the supported explanation: Correct its role to validation; it was used for selection. Changing the label on a dataset does not make its **evidence** untouched.

Repair and retain the record

Review Efficiency and resource measurement (ai-5/31). Goal: Calculate throughput and amortised time per example from repeated timings. Short practice: Why is 10 ms/case not necessarily a single-request latency?

Point to the relevant step in the picture, then write the changed result and the rule or **evidence** that supports it.. Return to this week and explain what you changed.

Review Uncertainty and failure analysis (ai-5/29). Goal: Distinguish a predicted class, confidence score and observed correctness. Short practice: What is C's positive-class probability and chosen-label confidence?

Point to the relevant step in the picture, then write the changed result and the rule or **evidence** that supports it.. Return to this week and explain what you changed.

Review Embeddings and neighbourhoods (ai-5/26). Goal: Look up a vector using a token ID and preserve sequence shape. Short practice: What would justify claiming nearby vectors mean related topics?

Point to the relevant step in the picture, then write the changed result and the rule or **evidence** that supports it.. Return to this week and explain what you changed.

The browser lab lets you edit the supported Python examples already included in this lesson. Predict first, run, inspect the actual **output** or error, change one **input**, then run again.

Keep code, **prediction**s, actual results and a reasoned revision together. On paper, label your result as a trace rather than an execution.

Use Download completed work in the hosted lesson to export your answers, reflections, practice feedback and code-run **evidence**. Open the HTML copy to read or print it.

Drafts stay in this browser when storage is available; clear drafts on a shared device.

<details>
<summary>Earlier wording, alternatives and picture transcripts</summary>

Use this supported practice once, in the browser or on paper. Keep the reserved independent assessment for a separate first attempt before teacher feedback.

Repair a missing building block

Predict, run and debug

Keep your completed work

</details>

<details>
<summary>Reference · Six-picture concept journey</summary>

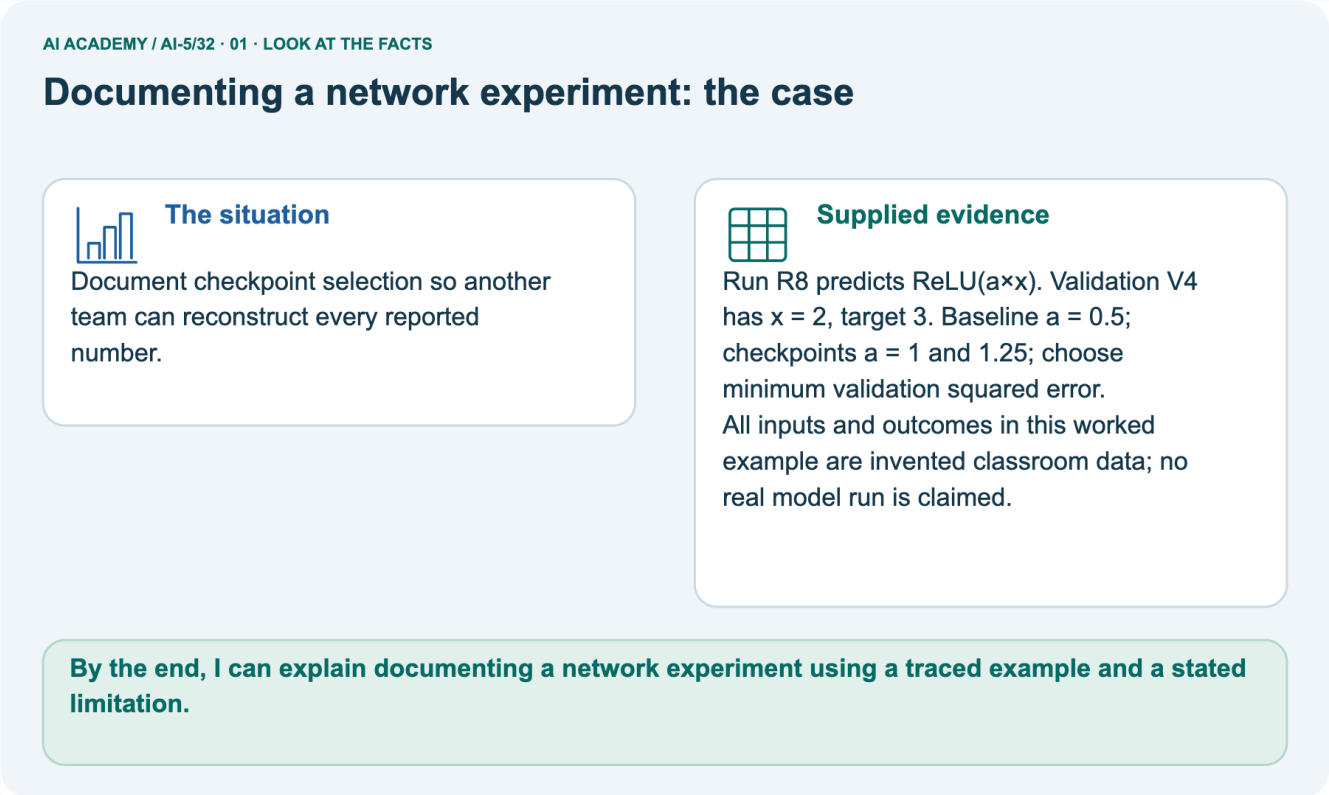
</details>

REFERENCE · SECTION 18

Reference · Six-picture concept journey

> Choose the picture that supports the current explanation.

01. Documenting a network experiment: the case



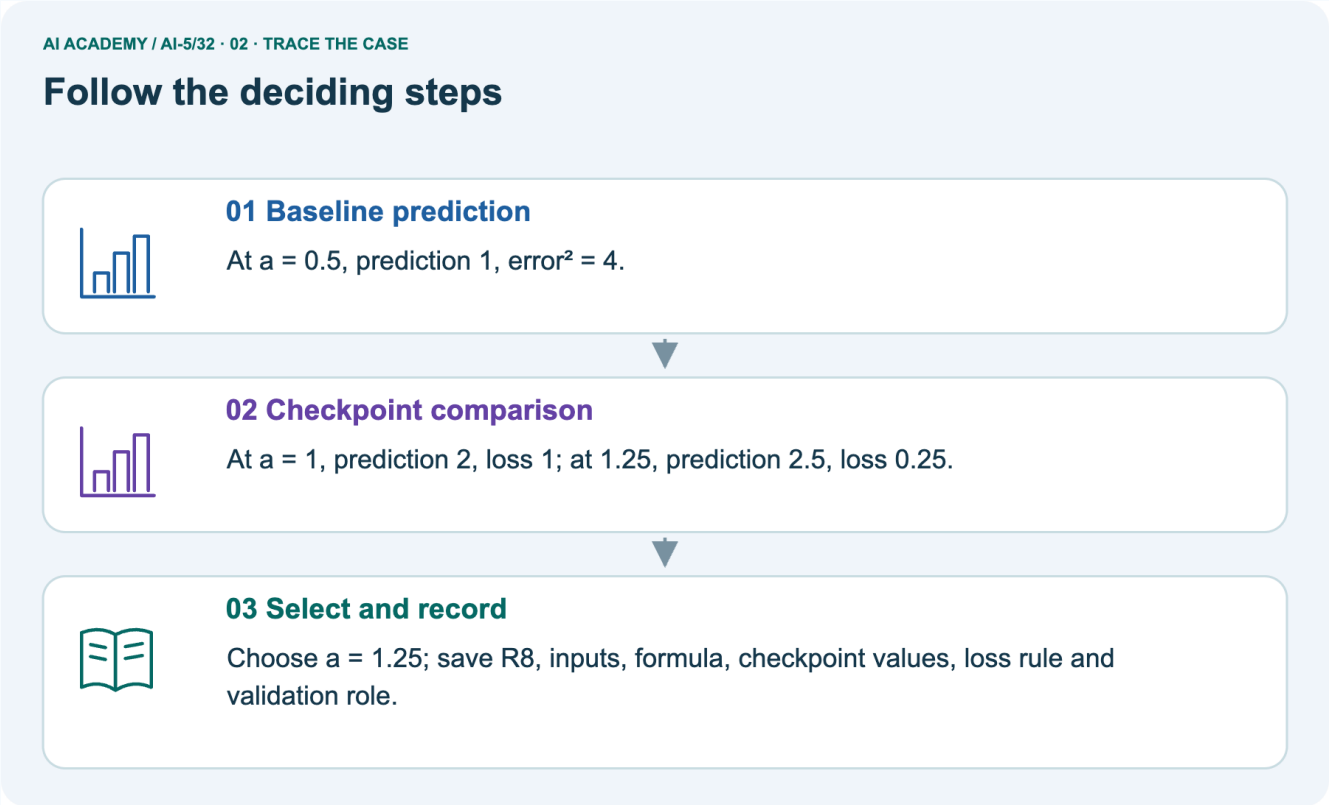
Follow the labelled picture with the matching explanation. Open the vector diagram

Document **checkpoint** selection so another team can reconstruct every reported number. Supplied facts: Run R8 predicts ReLU(a×x). Validation V4 has x = 2, target 3.

Baseline a = 0.5; **checkpoint**s a = 1 and 1.25; choose minimum validation squared error. All **input**s and outcomes in this worked example are invented classroom data; no real model run is claimed.

Point to the given fact that matters most. Explain why it matters.

02. Follow the deciding steps

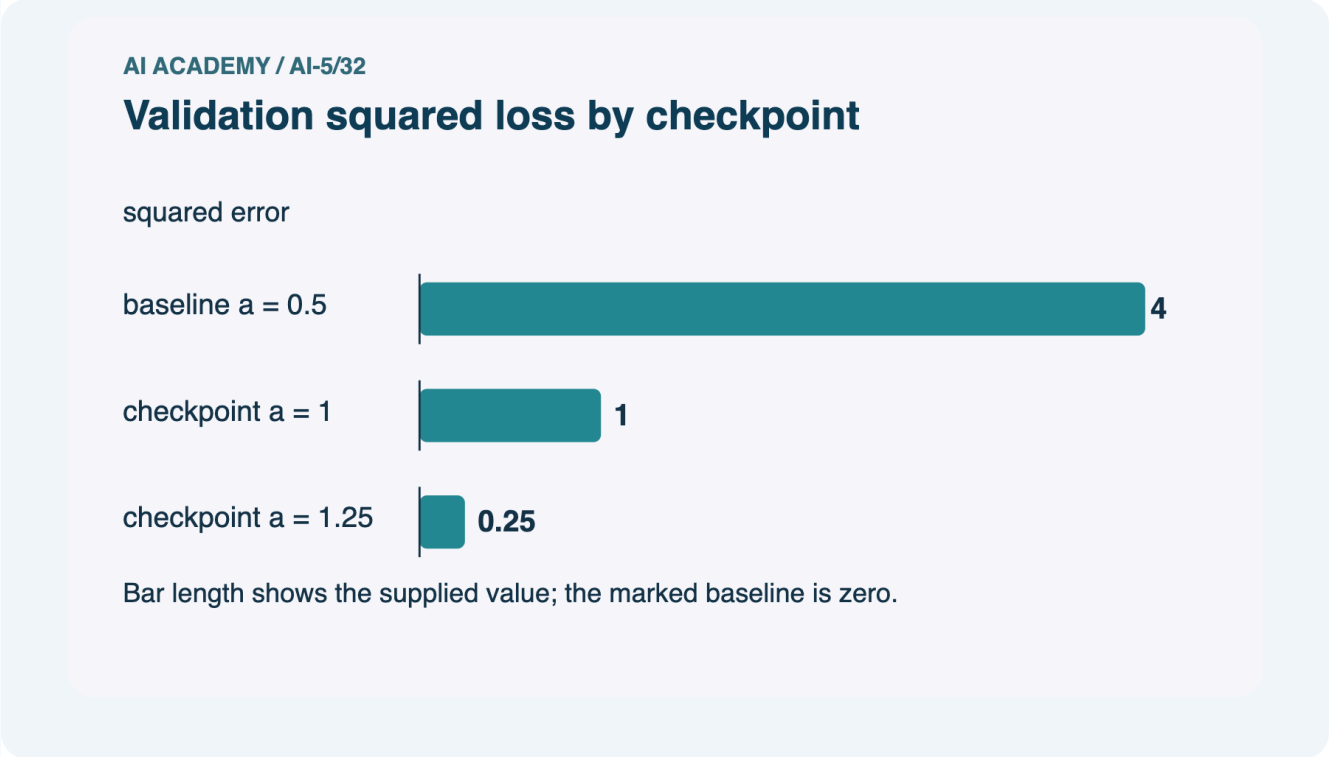


Follow the labelled picture with the matching explanation. Open the vector diagram

Baseline **prediction**: At a = 0.5, **prediction** 1, error² = 4. → **Checkpoint** comparison: At a = 1, **prediction** 2, loss 1; at 1.25, **prediction** 2.5, loss 0.25. → Select and record: Choose a = 1.25; save R8, **input**s, formula, **checkpoint** values, loss rule and validation role.

Point to each numbered stage and explain which recorded **input** it uses.

03. Validation squared loss by checkpoint

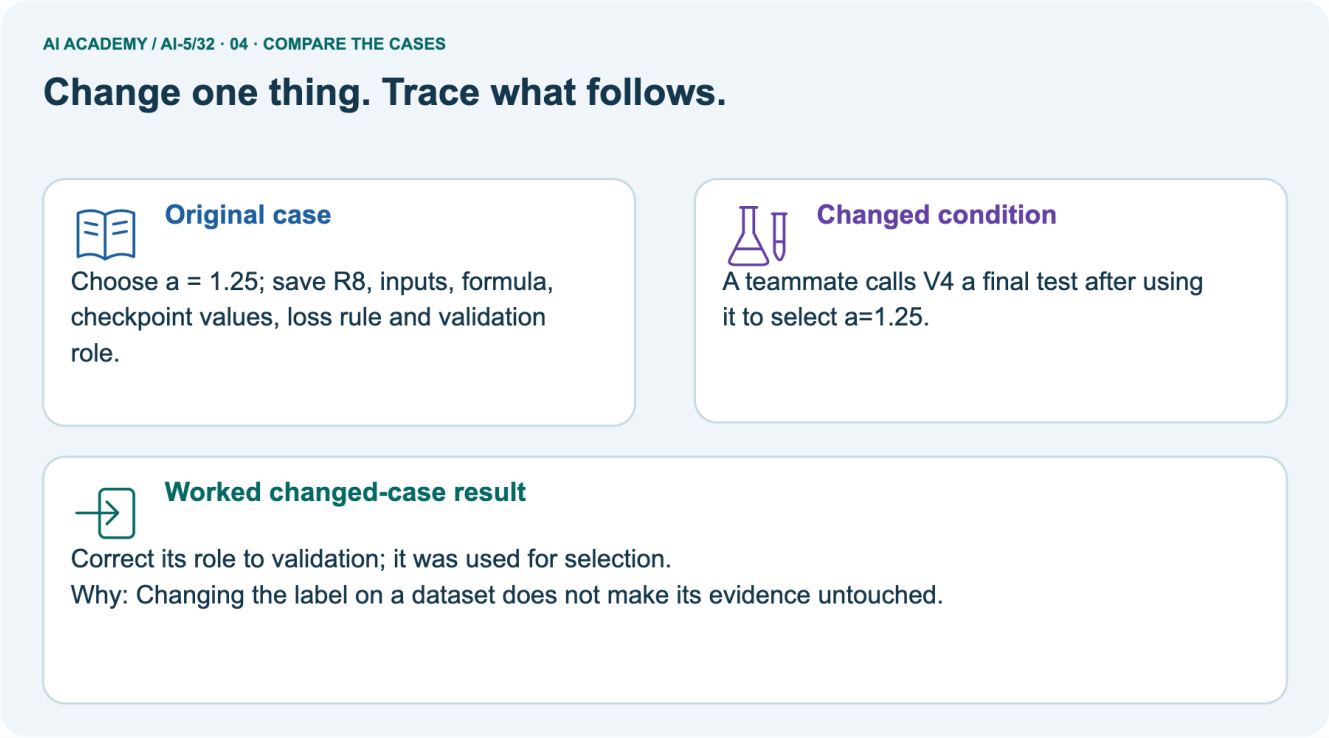


Follow the labelled picture with the matching explanation. Open the vector diagram

Validation squared loss by **checkpoint**. Units: squared error. baseline a = 0.5: 4. **checkpoint** a = 1: 1. **checkpoint** a = 1.25: 0.25.

Read the labels and values. Explain the relationship the model shows.

04. Change one thing. Trace what follows.



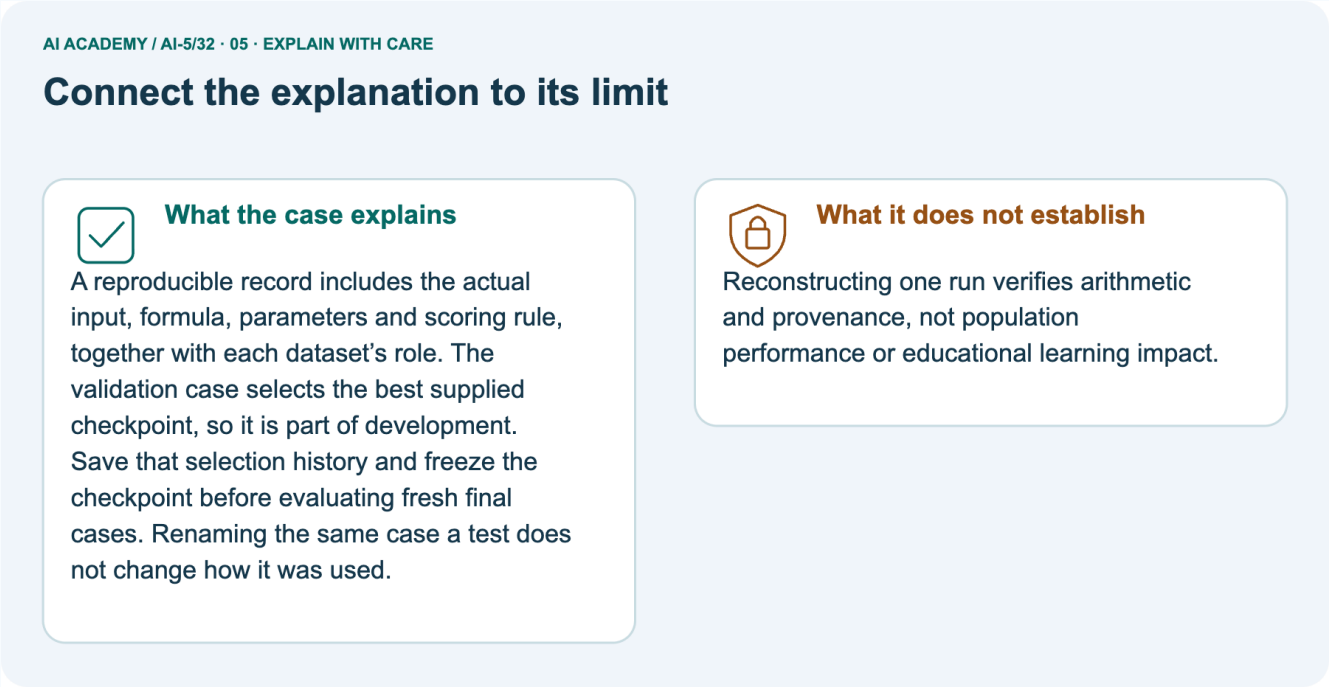
Follow the labelled picture with the matching explanation. Open the vector diagram

Original case: Choose a = 1.25; save R8, **input**s, formula, **checkpoint** values, loss rule and validation role. Changed condition: A teammate calls V4 a final test after using it to select a=1.25.

Expected result: Correct its role to validation; it was used for selection. Reason: Changing the label on a dataset does not make its **evidence** untouched.

Predict before checking the worked changed-case result. Name what changed and what the supplied facts allow you to conclude.

05. Connect the explanation to its limit



Follow the labelled picture with the matching explanation. Open the vector diagram

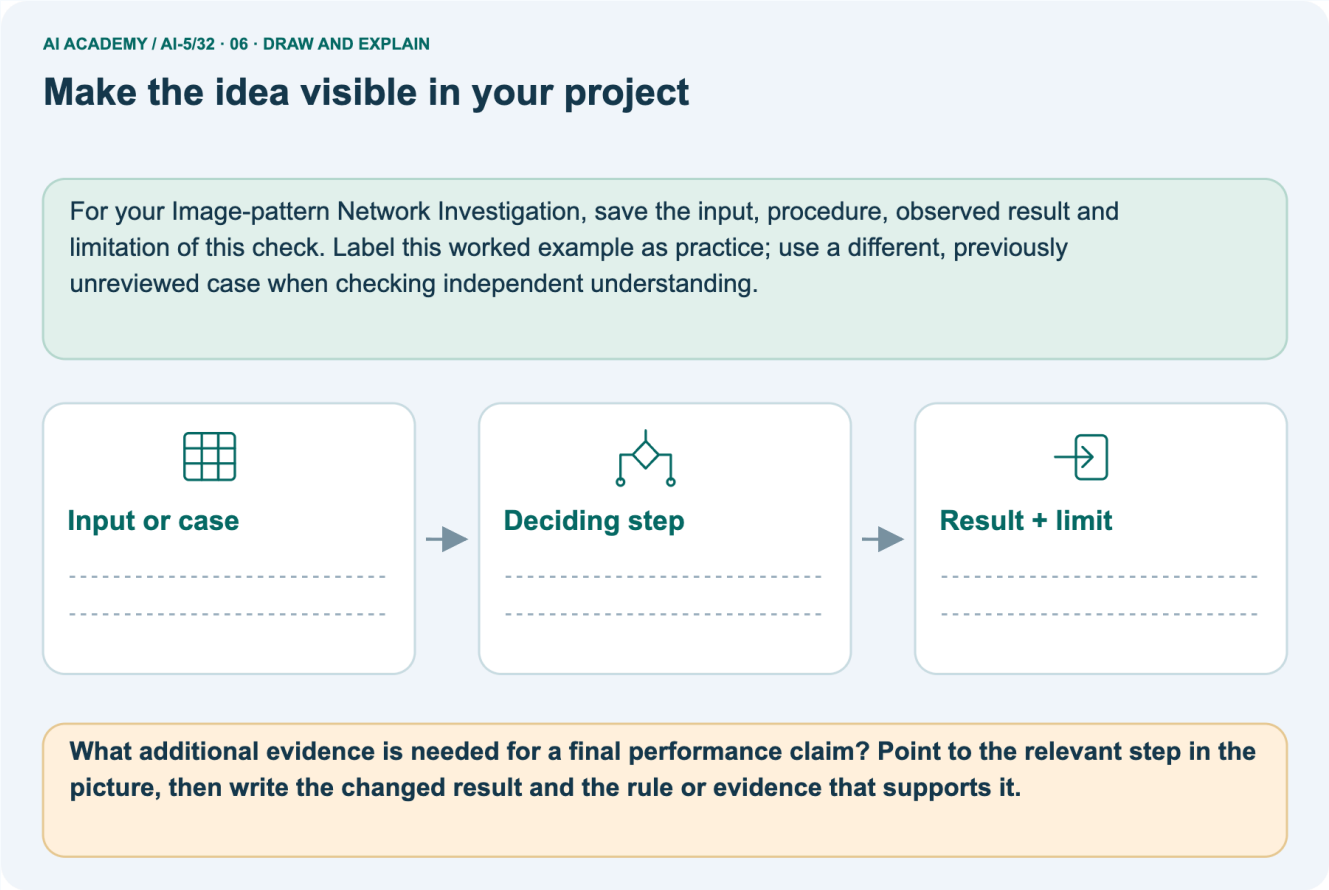
A reproducible record includes the actual **input**, formula, parameters and scoring rule, together with each dataset's role. The validation case selects the best supplied **checkpoint**, so it is part of development.

Save that selection history and freeze the **checkpoint** before evaluating fresh final cases. Renaming the same case a test does not change how it was used.

Limit: Reconstructing one run verifies arithmetic and provenance, not population performance or educational learning impact.

Explain the case, then point to a limit on the claim.

06. Make the idea visible in your project



Follow the labelled picture with the matching explanation. Open the vector diagram

For your Image-pattern Network Investigation, save the **input**, procedure, observed result and limitation of this check.

Label this worked example as practice; use a different, previously unreviewed case when checking independent understanding. Draw your **input**, deciding step, result and limit.

Supported question: What additional **evidence** is needed for a final performance claim? Point to the relevant step in the picture, then write the changed result and the rule or **evidence** that supports it.

Draw your project trace and justify the deciding step using **evidence**.

▼ Earlier wording, alternatives and picture transcripts

01. Documenting a network experiment: the case

02. Follow the deciding steps

03. Validation squared loss by checkpoint

04. Change one thing. Trace what follows.

05. Connect the explanation to its limit

06. Make the idea visible in your project

▼ Optional interactive practice and completed work

Save or print your responses for your portfolio. Progress and drafts on the hosted website stay in the current browser. Use only fictional or teacher-approved data.